FDI Analytics

Signal Intelligence Toolbox

User Guide

for Operators and General Users

No programming or data-science background required.

This guide explains every screen, button, and setting in the toolbox in plain language — written for plant, process, and maintenance personnel who work with the application through its web interface only.

Contents

Welcome

FDI Analytics is a point-and-click toolbox for **Fault Detection and Identification (FDI)** on industrial sensor data — vibration, temperature, pressure, current, or any other measurement that is logged over time.

It answers two practical questions for a piece of equipment or process:

1. **Is something unusual happening right now?** (Fault *Detection*)
2. **Which measured variable looks most responsible for it?** (Fault *Identification*)

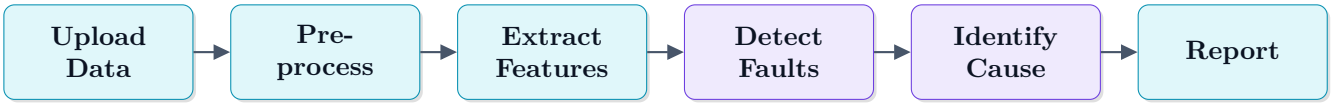
It does this **without needing historical examples of failures**. You do not need to tell the toolbox what a "bad" reading looks like in advance — it learns what *normal* looks like directly from the data you give it, and then flags whatever doesn't fit that pattern. This is sometimes called *unsupervised* analysis, and it is well suited to industrial settings where actual failure data is rare (which is exactly why it's hard to collect).

Who this guide is for

Anyone who opens the toolbox in a web browser and works with the buttons, menus, and charts on screen. No knowledge of Python, statistics, or machine learning is assumed — every technical term used by the interface is explained here in plain language, with a full glossary in Section 13 for quick lookup.

The six-stage workflow

The toolbox walks you through six stages, shown below. You move through them left to right using the navigation sidebar; each stage's output feeds the next one automatically.



Stage	What happens
Upload Data	You load a spreadsheet of sensor readings (CSV or Excel).
Pre-process	Optional clean-up: fill gaps, remove spikes, smooth noise.
Extract Features	The raw signal is summarized into a handful of descriptive numbers per time window.
Detect Faults	The toolbox learns what "normal" looks like and flags windows that deviate from it.
Identify Cause	For each flagged window, the toolbox ranks which variables/features look most abnormal.
Report	A summary you can download as a spreadsheet or a printable report.

NOTE

Every stage offers two "flavors" of method: a **Traditional** one (classic statistics and machine learning) and a **Deep Learning** one (a small neural network that learns its own patterns from your data). You choose which to use with a single switch at the top of each page — see Section 5.2.

Getting Started

Opening the toolbox

The toolbox runs as a small web application. Whoever set it up for you (an engineer or IT colleague) will give you an address that looks like:

`http://127.0.0.1:8050`

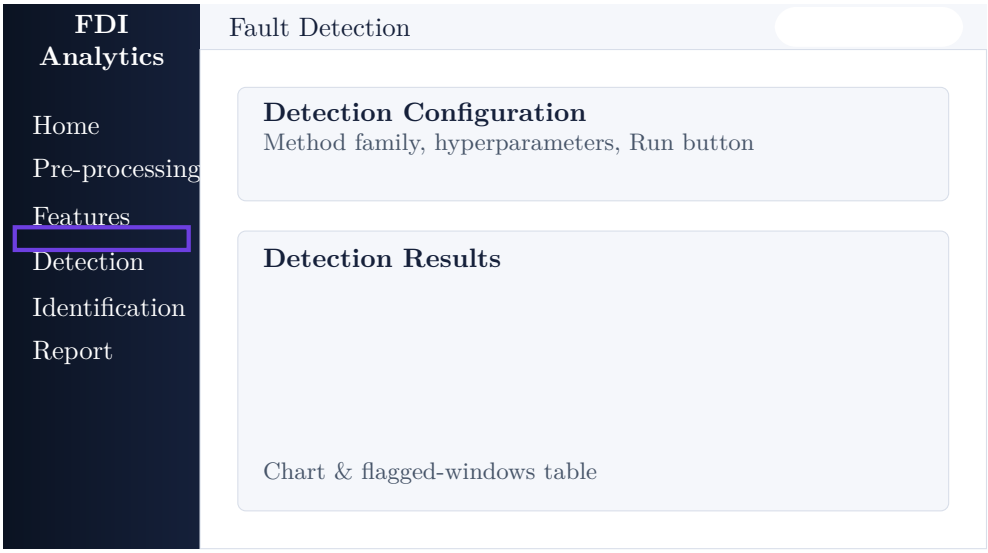
Paste this into a normal web browser tab — Chrome, Edge, or Firefox all work well.

CAUTION

Do *not* open the link inside a code editor's built-in preview window (for example VS Code's "Simple Browser"). Those mini-browsers cannot display the toolbox correctly. Always use a regular browser window or tab.

The screen layout

Every page of the toolbox shares the same frame, shown schematically below.



Sidebar (navigation) *Main content (cards for this page)*

- **Sidebar (left)** — the six workflow stages. Click any item to jump to that page. The current page is highlighted. A small arrow button at the bottom of the sidebar collapses it to icons only, useful on smaller screens.
- **Top bar** — shows which page you're on, a small status chip telling you whether data is loaded / detection has run, and the **AI Assistant** button, always available on the right regardless of which page you're viewing (see Section 9).
- **Main content** — the actual controls and results for the current page, organized into rounded "cards."
- **Back / Next buttons** — at the bottom of each page, these move you to the neighboring workflow stage, so you can follow the recommended order without reaching for the sidebar.

TIP

Your uploaded data and every result you generate stay loaded as you move between pages, so you can freely go back and forth — for example, change a setting on the Pre-processing page and re-check the Feature Extraction page without losing anything. If you are ever unsure how far along you are, glance at the status chip in the top bar.

Step 1 — Home: Loading Your Data

What your file should look like

The toolbox reads ordinary spreadsheet files. Each **column** is one measured variable, and each **row** is one reading in time, in order from oldest to newest. The first row holds the column names.

vibration	temperature	pressure	...
0.42	38.1	2.05	...
0.45	38.0	2.06	...
0.41	38.2	2.04	...
... one row per time sample, in time order ...			

NOTE

You do *not* need a column that marks which rows are "faulty." Discovering that is the whole point of the toolbox. If you happen to have such a column, it is simply carried along and ignored by the analysis.

Uploading a file

Open the **Home** page (the house icon, top of the sidebar). You'll see an upload area.

1. Drag your file onto the dashed box, or click it to browse your computer. Supported file types: `.csv`, `.xls`, `.xlsx`.
2. A green confirmation banner reports the number of rows and columns found, and the top-bar chip switches to **Dataset loaded**.
3. Check the **preview table** of the first 10 rows to confirm the right file loaded correctly.
4. Use the **Variable Explorer** chart — pick one or more numeric columns from the drop-down to plot them and visually sanity-check the data (e.g. spot an obviously broken sensor before going further).

CAUTION

Only **numeric** columns (numbers) are used by the rest of the toolbox. Text columns (e.g. a timestamp written as text, or an operator name) are shown in the preview but ignored by every later stage. If your timestamp is important, make sure your data is already sorted in time order before uploading — the toolbox treats row order as time order.

Step 2 — Pre-processing

Real sensor data is rarely perfectly clean: there can be missing readings, occasional spiky errors, slow drift, and background noise. The Pre-processing page lets you fix these *before* the toolbox starts looking for faults — which makes detection more reliable. **Every step here is optional.** If your data is already clean, you can skip this page entirely.

The five clean-up steps

The steps are applied in this fixed order, and you can switch each one on or off independently:

1. **Missing Value Handling** — what to do about gaps in the data (e.g. a sensor briefly dropped out). Options: drop those rows, or fill the gap using the average, the middle value, or a smooth interpolation between neighboring readings.
2. **Outlier Removal** — catches and tames extreme spikes that are almost certainly sensor glitches rather than real behavior, using a statistical "how far from typical" threshold you control.
3. **Detrending** — removes slow, long-term drift (for example, a temperature sensor that gradually creeps upward over hours for reasons unrelated to a fault) so the analysis focuses on short-term behavior.
4. **Noise Filtering / Smoothing** — evens out small, high-frequency jitter, similar to taking a rolling average.
5. **Normalization / Scaling** — rescales every variable onto a comparable range, which matters because raw units differ wildly (e.g. temperature in tens of degrees vs. vibration in tiny fractions) and later stages compare variables to one another.

How to use the page

1. Click a step's title bar to expand its settings.
2. Tick **Enable this step** to turn it on.
3. Choose a method from the list, and set its numeric parameter if there is one (a label tells you what the number controls, e.g. a window size or a sensitivity threshold).
4. Repeat for any other steps you want.
5. Click **Apply Pre-processing**.

While it runs, a progress bar appears showing exactly which of the five steps is currently being applied, with a **Cancel** button beside it — see Section 10 for how progress and cancelling work everywhere in the toolbox.

Afterwards, scroll down to **Before vs. After Comparison**: pick a variable from the drop-down to see the original signal (grey) overlaid with the cleaned-up version (teal), so you can visually confirm the cleanup did what you expected.

TIP

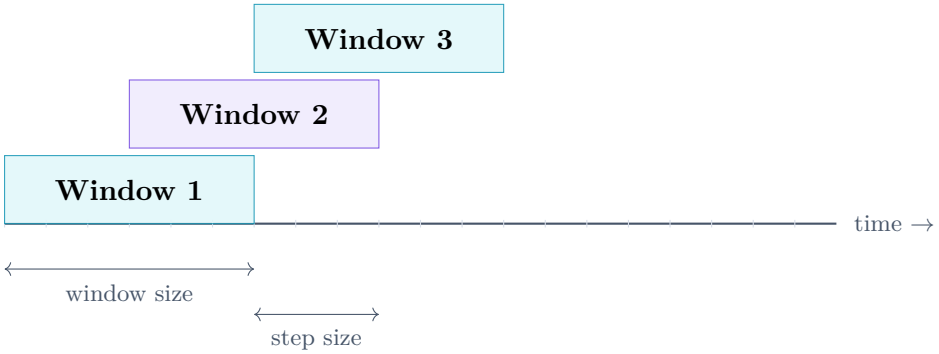
If you're not sure what to enable, a reasonable starting point for noisy industrial data is: **Outlier Removal** (Z-score) and **Noise Filtering** (Moving Average) only. Add the others if you can see a specific problem (gaps, drift) in the Variable Explorer chart from the Home page.

Step 3 — Feature Extraction

A raw sensor signal is just a long list of numbers, second by second. Before the toolbox can spot anomalies, it needs to summarize short **windows** of that signal into a handful of descriptive numbers — this is called *feature extraction*.

Sliding windows, in plain terms

The toolbox slides a fixed-size "viewing window" along your data, step by step, and computes a summary for everything inside the window each time it moves.



- **Window size** — how many consecutive readings go into one summary. Bigger windows give smoother, more stable summaries but react more slowly to short events; smaller windows react faster but are noisier.
- **Step size** — how far the window moves before computing the next summary. A step size smaller than the window size means windows overlap (more, smoother summaries); a step size equal to the window size means no overlap.
- **Sampling frequency** — how many readings per second your data represents. Only used by a couple of the traditional frequency-based features below; leave at the default of `1` if you're not sure.

Method Family: Traditional vs. Deep Learning

At the top of the page is a switch between two ways of summarizing each window.

Traditional — statistical & frequency-domain features

Tick the features you want computed for every window, for every selected variable. In plain language:

Feature	What it tells you
Mean	The average level of the signal in the window.
Standard Deviation	How spread out / jittery the readings are.
RMS	A size measure that emphasizes larger swings (commonly used for vibration severity).
Variance	Spread, before taking a square root (same idea as Std. Dev.).
Skewness	Whether the readings lean more above or below the average.
Kurtosis	How "spiky" the distribution is — high values mean occasional sharp peaks.
Peak-to-Peak	The gap between the highest and lowest reading in the window.
Crest Factor	How extreme the biggest peak is relative to the overall signal size — a classic early-warning indicator in vibration monitoring.
Dominant Frequency	The most prominent repeating rate/frequency in the window.
Spectral Energy	The total "vibration energy" across all frequencies in the window.

Deep Learning — learned embeddings

DEEP LEARNING

Instead of hand-picking statistics, a small neural network (an *autoencoder*, explained in the Glossary) is trained, per variable, to compress each window down to a short list of numbers and then reconstruct the original window from them. Those compressed numbers — called the *latent* representation — become the features. The idea is that the network discovers whatever pattern best describes *your* specific signal, rather than relying on a fixed, generic list of statistics.

Three architectures are offered:

- **Dense Autoencoder** — looks at the whole window at once; a good general-purpose default.
- **LSTM Autoencoder** — pays attention to the *order* of points in time, useful if the timing/sequence of events within a window matters.
- **1D-CNN Autoencoder** — looks for small repeating local shapes in the waveform, similar to how it might spot a recurring "blip" pattern.

You also choose:

- **Latent dimension** — how many compressed numbers describe each window. Fewer numbers force a tighter summary; more numbers can capture more detail but need more data to train reliably.
- **Training epochs** — how many times the network studies your data while learning. More epochs can improve quality but take longer — watch the progress bar.

How to use the page

1. Choose your **Data Source** — **Pre-processed data** (recommended, if you ran Step 2) or **Raw data**.
2. Pick the **Variables** you want to extract features from (you can select several).
3. Select the **Method Family** — Traditional or Deep Learning — and its settings: tick the features you want, or pick the network architecture and set the latent dimension / training epochs.
4. Set the **Window parameters** — window size, step size, and sampling frequency.
5. Click **Extract Features**. A progress bar tracks the work (window-by-window for traditional features, epoch-by-epoch per variable for deep learning).

When it finishes, the results appear as a table (downloadable as CSV) and a chart of any one feature over time — handy for watching the moment a feature changes.

Step 4 — Fault Detection

This is where the toolbox decides which time windows look **abnormal**. It does this by learning the overall "shape" of your data and flagging whatever doesn't fit — again, with no example faults required up front.

The Contamination setting

Every detection method — traditional or deep learning — uses one shared setting:

NOTE

Contamination is your best estimate of *what fraction of the time* the equipment is behaving abnormally, expressed as a number between `0.01` (1%) and `0.5` (50%). It directly controls how trigger-happy the alarm is:

- **Lower** contamination → fewer, more confident alarms.
- **Higher** contamination → more alarms, including borderline ones.

A typical starting point is `0.05` (5%). If nothing gets flagged, raise it; if everything gets flagged, lower it.

There is also a **Standardize features** toggle (on by default and recommended) — it rescales every feature onto a comparable range before comparing them, which matters because, like in pre-processing, raw feature units differ wildly.

Traditional algorithms

Method	Plain-language idea
PCA (Hotelling's T^2)	Finds the main directions in which your data normally varies, then flags windows that fall far outside that normal "cloud" shape.
Isolation Forest	Repeatedly tries to isolate each window from the rest with random splits; genuinely unusual windows are easy to isolate quickly, normal ones are not.
One-Class SVM	Draws a tight boundary around the bulk of "normal" windows; anything outside the boundary is flagged.
Local Outlier Factor (LOF)	Compares each window to its nearest neighbors; if a window sits in a much sparser neighborhood than typical, it's flagged.

Each method has one or two extra dials specific to it (e.g. *number of neighbors* for LOF) — defaults are sensible; only change them if you know what you're adjusting.

Deep learning algorithms

DEEP LEARNING

Method	Plain-language idea
Autoencoder	A small network learns to reconstruct normal windows accurately. Windows it reconstructs *badly* are flagged as abnormal.
Variational Autoencoder (VAE)	A more probability-aware version of the above; an extra **KL weight (β)** dial balances reconstruction accuracy against how "tidy" the learned representation is.
Deep SVDD	The network learns to squeeze all normal windows into a tight imaginary ball; windows that land far from the center of that ball are flagged.

Shared settings: **Latent dimension** and **Training epochs** (same meaning as in Feature Extraction).

How to use the page

1. Choose the **Method Family** — Traditional or Deep Learning.
2. Pick the specific **algorithm** from the list (the default is a sound first choice).
3. Set the **Contamination** (sensitivity) value, and leave **Standardize features** switched on.
4. Adjust any algorithm-specific dials if you need to — the defaults are sensible, so you can usually leave them alone.
5. Click **Run Fault Detection**. Watch the progress bar (per-epoch for deep learning, staged checkpoints for traditional methods); cancel any time with the button beside it.

Results show as:

- A chart of every window's **anomaly score** over time, with flagged windows marked as red crosses and a dashed threshold line.
- A table listing every flagged window and its score.

Step 5 — Fault Identification

Detection tells you *when* something looks wrong. Identification tells you *which variable, and which feature of that variable*, looks most responsible — a strong first clue for root-cause investigation.

Traditional: Z-score baseline

This method compares every flagged window's features against the **average behavior of the windows that were *not* flagged** (i.e. against "normal"). The further a feature sits from its normal average (in standardized units called a *Z-score*), the more it contributes to that window being flagged. This method is instant and runs automatically — no button needed.

Deep Learning: Autoencoder-based methods

DEEP LEARNING

- **Autoencoder Residual** — trains a small autoencoder on the data and looks at *which specific features* it fails to reconstruct well. Features the network struggles with are the ones driving the abnormality.
- **Gradient × Input Saliency** — asks the trained network "which features, if nudged slightly, would change the abnormality score the most?" Features with a bigger effect are ranked higher. This is a common technique for explaining what a neural network is paying attention to.

Because training a network takes a moment, this method requires you to click **Train & Run Deep Identification** explicitly (with its own progress bar and Cancel button). The result is then cached, so switching between flagged windows below is instant — it will not retrain.

CAUTION

If you re-run **Detection** with different settings, any cached deep learning identification result is automatically cleared, since it would no longer match the new set of flagged windows. You'll need to click **Train & Run Deep Identification** again.

How to use the page

1. Make sure you have already run **Detection** — this page works from its flagged windows.
2. Read the **Variable Contributions** and **Top Contributing Features** charts. With the Traditional (Z-score) method these appear automatically — no button needed.
3. To use a Deep Learning method instead, select it and click **Train & Run Deep Identification**, then wait for the progress bar.
4. Use the **Window Drill-down** drop-down to inspect any single flagged window in detail.

Reading the results

- **Variable Contributions** — a bar chart ranking your original sensor variables by how implicated they are, averaged across all flagged windows.
- **Top Contributing Features** — the same idea at a finer level (e.g. "vibration — Crest Factor" specifically, not just "vibration").

- **Window Drill-down** — pick one specific flagged window from the drop-down to see exactly which features drove *that* alarm, with both a chart and a table of raw values.

Step 6 — Evaluation & Report

This page summarizes everything from your most recent detection run and lets you take results outside the toolbox.

- **Detection Summary** — key numbers at a glance: total windows, how many were flagged, the fault rate, score threshold, and score statistics.
- **Anomaly Score Distribution** — a histogram comparing normal vs. flagged windows, so you can see how cleanly separated they are.
- **Feature Deviation Heatmap** — a color grid of every flagged window against its most deviating features; reds and blues show the direction and strength of each deviation.
- **Top Implicated Variables** — the same ranking shown on the Identification page, here for the full record.

How to use the page

1. Review the **Detection Summary** numbers at the top — total windows, how many were flagged, and the fault rate.
2. Scan the **score distribution**, the **deviation heatmap**, and the **top implicated variables** to judge how clear-cut the result is.
3. Click **Download CSV** for the raw data, or **Download HTML Report** for a formatted, printable summary.

Downloading results

- **Download CSV** — every window's score and flag status, plus full feature detail for flagged windows, as a spreadsheet you can open in Excel or share with colleagues.
- **Download HTML Report** — a formatted, printable report with the same summary numbers and tables, suitable for attaching to a maintenance ticket or email.

The AI Assistant

The violet **AI Assistant** button, always visible in the top-right corner, opens a chat assistant that already knows the current state of your analysis — what data is loaded, what's been extracted, and what's been detected.

How to use the page

1. Open **API Configuration** and pick a provider (see the options below). This one-time setup connects the assistant to an AI service.
2. Enter the API key and/or base URL as required, then click **Save Configuration**.
3. Click a **Quick Prompt** button, or type your own question in plain English, to start the conversation.
4. Use **Clear** whenever you want to begin a fresh conversation.

Setting it up (optional, one time)

The assistant needs to be connected to an AI provider. Open **API Configuration** and choose one:

- **OpenAI** or **Anthropic (Claude)** — you'll need an API key from that provider (ask your IT/engineering team if your organization already has one).
- **Ollama** — a locally installed model; no key needed, but it must already be running on the same computer.
- **Custom endpoint** — for an internal/company AI service.

Click **Save Configuration** once filled in.

> **NOTE**
>
> The key is kept only in your own browser/session for talking directly to the AI provider — it is not stored on, or sent to, any server belonging to this toolbox.

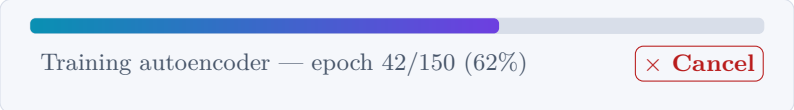
Using it

- Click any **Quick Prompt** button (e.g. "Interpret detection results," "Recommend preprocessing") to ask a ready-made question instantly.
- Or type your own question in plain English — e.g. *"Why was window 140 flagged?"* or *"Which preprocessing should I try next?"*
- Use **Clear** to start a fresh conversation at any time.

The assistant is a thinking aid for interpreting results — it does not change any data or settings on your behalf.

Progress Bars and Cancelling

Every action that takes real time — **Apply Pre-processing**, **Extract Features**, **Run Fault Detection**, and **Train & Run Deep Identification** — shows the same kind of live progress bar once you click it.



- For **deep learning** steps, the bar reports real training progress: *"epoch 42 of 150."*
- For **traditional** steps, the bar reports real checkpoints of the work (e.g. *"window 1,400 of 3,000"* during feature extraction, or *"Fitting Isolation Forest"* → *"Scoring windows"* during detection).
- Click **Cancel** at any time to stop the action immediately. The page returns to its previous state — nothing partial is saved.

CAUTION

If you never see a progress bar at all and the page instead shows a plain "Updating…" message, an optional add-on package (`diskcache`) is missing from the installation. The toolbox still works correctly either way — ask your IT/engineering contact to install it if you'd like live progress and the Cancel button.

Practical Tips for Operators

- **Start traditional, escalate to deep learning.** Traditional methods are fast and give a good first read. Reach for Deep Learning when traditional results look unclear, or when you suspect a subtler pattern that simple statistics might miss.
- **Sanity-check with the Variable Explorer first.** Before doing anything else, plot your raw variables on the Home page — a broken sensor or an obvious data problem is much easier to spot by eye than to diagnose later in the pipeline.
- **Tune Contamination, not the algorithm, first.** If your alarm count looks wrong, adjust Contamination (Section 6.1) before switching algorithms — it's the most direct lever.
- **Always check Identification before acting.** A flagged window is a starting point for investigation, not a confirmed fault. Use the Identification page to see *why* it was flagged before raising a maintenance action.
- **Keep window size physically meaningful.** Choose a window size that corresponds to a sensible real-world duration for the phenomenon you're watching for (e.g. long enough to contain a full rotation cycle for vibration data).
- **Re-run, don't second-guess.** Because every step is fast to re-run (and now cancellable), it's almost always faster to try a different setting and re-run than to try to mentally work out the effect in advance.

Troubleshooting & FAQ

Clicking a button does nothing visible.

Check for a small amber banner near the button — it usually means an optional component (PyTorch for deep learning, or diskcache for progress bars) isn't installed. Everything else on the page still works; contact your IT/engineering team to add the missing piece if you need that specific feature.

The data preview looks scrambled (everything in one column, odd symbols).

This is almost always the file format, not the toolbox. Re-export your data as a standard comma-separated `.csv` with the column names in the first row, then upload again. Semicolon-separated exports and unusual text encodings are the usual culprits.

No windows get flagged on the Detection page.

Raise the **Contamination** value (Section 6.1), or try a different detection algorithm.

Almost everything gets flagged.

Lower **Contamination**. Also check that **Standardize features** is switched on, and consider revisiting Pre-processing — unremoved outliers or drift can make the entire dataset look "abnormal."

The page looks broken / blank / unstyled.

Make sure you're using a normal browser tab (Chrome, Edge, Firefox), not an embedded preview window inside another application.

Deep learning steps are taking a long time.

Watch the progress bar to see real-time status, and use **Cancel** if needed. To speed things up next time: reduce the number of **Training epochs**, use a smaller **Latent dimension**, select fewer variables, or switch to a Traditional method for a quicker first look.

I changed a setting but nothing on screen updated.

Most pages update automatically, but the steps with a visible **Run**/**Extract**/**Apply** button require you to click it again after changing a setting — this is intentional, since those steps can take real time.

I want to start over completely.

Reload the page in your browser and upload your file again on the Home page.

Glossary

Anomaly / Fault
A time window whose features look meaningfully different from the toolbox's learned notion of "normal." Not necessarily a confirmed mechanical failure — a starting point for investigation.

Anomaly Score
A single number summarizing how unusual a window is. Higher generally means more unusual; the exact scale depends on the method used.

Autoencoder
A small neural network trained to compress its input down to a few numbers and then reconstruct the original input from them. Used both to create features and to detect anomalies (badly-reconstructed inputs are flagged).

Contamination
Your estimate of what fraction of windows are abnormal; controls detection sensitivity. See Section 6.1.

Deep Learning
A family of methods using neural networks — layered mathematical functions that learn patterns directly from data, rather than following a fixed, hand-written formula.

Feature
A single descriptive number computed from a window of raw data (e.g. its average, or a learned "latent" number from an autoencoder).

Hyperparameter
A setting you choose before running a method (e.g. Contamination, Latent dimension, Training epochs) as opposed to something the method learns automatically from the data.

Isolation Forest
A traditional detection algorithm that flags windows that are unusually easy to separate from the rest of the data using random splits.

Latent dimension
The number of compressed values an autoencoder uses to summarize its input. Smaller forces a tighter summary.

Local Outlier Factor (LOF)
A traditional detection algorithm that flags windows sitting in a much sparser "neighborhood" of similar windows than is typical.

Normalization / Standardization
Rescaling variables or features onto comparable ranges so that no single one dominates a comparison just because of its raw units.

One-Class SVM
A traditional detection algorithm that draws a boundary around the bulk of normal data; anything outside is flagged.

Outlier
A single reading (or short burst of readings) that is extreme compared to its surroundings — often a sensor glitch rather than real behavior, which is why Pre-processing can remove it.

PCA / Hotelling's T^2
Principal Component Analysis: a traditional technique that finds the main directions in which data normally varies, and flags windows that fall far outside that pattern.

Saliency (Gradient × Input)
A technique for asking a trained neural network which input features it is most "paying attention to" when producing its output.

Sliding window
A fixed-length snippet of consecutive readings, moved forward in fixed steps across the whole signal. See Section 5.

Threshold
The anomaly-score cutoff, set automatically from your Contamination value, above which a window is flagged.

Unsupervised
Learning "normal" directly from your data, without being given labeled examples of past faults.

VAE (Variational Autoencoder)
A probability-aware variant of an autoencoder, with an extra dial (KL weight, β) balancing reconstruction accuracy against the tidiness of its learned representation.

Z-score
How many standard deviations a value sits away from its average — the basis of the Traditional Identification method.

Quick Reference

Page	Main action button	What you get
Home	— (drag/drop or click upload)	Loaded dataset, preview, variable chart
Pre-processing	Apply Pre-processing	Cleaned dataset, before/after chart
Feature Extraction	Extract Features	Feature table (downloadable), feature-over-time chart
Fault Detection	Run Fault Detection	Anomaly score chart, flagged-windows table
Fault Identification	(automatic) / Train & Run Deep Identification	Variable & feature ranking, window drill-down
Evaluation & Report	Download CSV / Download HTML Report	Summary stats, charts, exportable report

Color	Meaning across the toolbox
	Data / normal signal, and primary actions
	Deep learning / AI Assistant features
	Success / completed
	Warning — check before proceeding
	Flagged fault / error

NOTE

The golden rule: work top-to-bottom down the sidebar, use the **Next** button to move along, and watch the top-bar status chip to confirm each stage is done. Start with **Traditional** methods for a fast result, and bring in **Deep Learning** only when you want a closer look.

End of document — FDI Analytics User Guide